

普及组初赛模拟卷 19 解析

(C++语言 1.5 小时完成)

一、单项选择题 (共 20 题, 每题 2 分, 共计 40 分, 每题有且仅有一个正确答案。)

1. 如果将 ASCII 码大于 100 的字母输出到屏幕上, 一共有 (B) 个
A. 21 B. 22 C. 23 D. 24

2. 下列不同进制下的小数, 最小的是 (C)
A. $(0.101)_2$ B. $(0.5)_8$ C. $(0.6)_{10}$ D. $(0.A)_{16}$

解析: A 选项 $(0.101)_2 = 5/8$, B 选项 $(0.5)_8 = 5/8$, D 选项 $(0.A)_{16} = 10/16 = 5/8$ 。

3. 已知千兆宽带的带宽为 1000Mbps, 那么用该宽带下载文件的最大下载速度约为 (B)
A. 10MB/s B. 125MB/s C. 1000MB/s D. 1GB/s

解析: 1000Mbps 指的是 $1000 \times 10^6 \text{bit/s}$, 转换为字节要除以 8。

4. 在 C++ 中, 下列几种数据类型的变量中存储容量最大的是 (D)
A. char B. int C. long long D. long double

解析: long double 相当于 pascal 语言中的 extended, 占 10 个字节, 符号位 1 位, 阶码 15 位, 尾数 64 位。

5. 下列各种说法中, 正确的是 (A)

- A. 计算机通过 ip 地址来访问网络中的其他设备
B. “三网融合”的“三网”指的是局域网、城域网和广域网
C. 常见的计算机网络操作系统包括 Unix、Linux、Android 等
D. 开源软件可以自由地进行复制、修改和传播, 不需要遵守任何协议

解析: “三网融合”的“三网”指的是计算机网络、移动通信网络和广播电视网络, Android 不是网络操作系统, 开源软件需要遵守“开放软件协议”。

6. 下列计算机软件中能够浏览网页的是 (B)。
A. Dev-cpp B. Firefox C. Flash Player D. Photoshop

解析: Firefox 是火狐浏览器, Flash Player 是动画播放软件, Photoshop 是图像编辑软件。

7. C++ 中若 a 数组 a 的初始值为 {0, 1, 2, 3}, 整数 b 的初始值为 2, 则执行语句 $a[++b]=0$ 后 a 和 b 的值分别为 (D)

- A. {0, 1, 2, 3} 2 B. {0, 0, 2, 3} 2 C. {0, 1, 0, 3} 3 D. {0, 1, 2, 0} 3

解析: ++b 先做加法, 后返回值, 因此原句等价于: $b++; a[b]=0;$

8. 若整型变量 x 的值为 15, 下列选项中, 则表达式 $x^{(x+1)}$ 的值为是 (D)
A. 7 B. 8 C. 15 D. 31

解析: ^ 表示异或运算, $x=15$, $x+1=16$, 故 $15^{16}=31$ 。

9. 表达式 $a*(b+c)/d$ 用后缀表达式表示为: (A)

- A. $abc+*d/$ B. $abcd+*/$ C. $a+b*c/d$ D. $/*a+bcd$

解析: 画出表达式树, 后缀表达式就是表达式树的后序遍历。

10. 下列哪种数据结构存在一对多的对应关系: (C)

- A. 队列 B. 栈 C. 树 D. 链表

11. 对序列【1,3,4,6,9,12,15,18,21】进行二分查找, 若查找失败, 即被查找的数不在序列之中, 最多需要进行 (C) 次查找

- A. 2 B. 3 C. 4 D. 5

解析: 可以画出二叉查找树求深度, 也可以直接套公式 $\lfloor \log_2 n \rfloor + 1$ 。

12. 对序列【3,2,6,5,1,8,4】进行选择排序(升降序及左右方向未知), 则在经过 2 轮排序之后, 序列的中间结果不可能为 (D)

- A. 1,2,6,5,3,8,4 B. 3,2,4,5,1,6,8 C. 3,8,6,5,4,2,1 D. 8,6,5,2,1,3,4

解析: 分别枚举升降序和向左向右四种情况即可。

升序从左向右排序: 1,2,6,5,3,8,4

升序从右向左排序: 3,2,4,5,1,6,8

降序从左向右排序: 8,6,2,5,1,3,4

降序从右向左排序: 3,8,6,5,4,2,1

因此答案选 D。

13. 下列哪种排序算法的时间复杂度和数据范围有关: (C)

- A. 冒泡排序 B. 插入排序 C. 计数排序 D. 归并排序

14. 某算法的计算时间函数 $T(n)$ 满足 $T(n)=T(n-1)+n$, 则该算法的时间复杂度为: (D)

- A. $O(\log_2^n)$ B. $O(n)$ C. $O(n\log_2^n)$ D. $O(n^2)$

解析: $T(n)=n+T(n-1)=n+n-2+T(n-2)=n+n-1+n-2+\dots+1=n(n+1)/2=O(n^2)$ 。

15. 若 a,b,c,d,e,f 依次入栈, 已知 a 第 3 个出栈, 则可能的出栈顺序一共有 (B) 种

- A. 5 B. 10 C. 15 D. 20

解析: a 第 3 个出栈, 因此 b 和 c 在 a 之前完成入栈和出栈, d,e,f 在 a 之后完成出栈和入栈, 我们设 n 个元素的出栈顺序数为 $f(n)$, 则答案为 $f(2)*f(3)=2*5=10$ 。

16. 某双向链表 a 的列表存储形式如下, 其中每组数据中第一个元素是数据, 第二个元素是前驱指针, 第三个元素是后继指针, 链表的头指针为 head, 则当前链表的前 3 个元素之和为 (D)

```
int[][] a = {{3,4,3},{6,-1,4},{2,4,3},{1,0,-1},{4,1,0}};  
int head = 1;
```

- A. 10 B. 11 C. 12 D. 13

解析: 我们虽然是双向链表, 但是我们可以只需要考虑后继指针, 起点是 $a[1]$, 数据为 $a[1][0]$ 等于 6, 后继为 4; 第二个节点值为 $a[4][0]$ 等于 4, 后继为 0; 第三个节点值为 $a[0][0]$ 等于 3。因此前三个元素之和等于 $6+4+3=13$ 。

17. 已知一棵二叉树一共有 2022 个节点, 根节点高度为 1, 则二叉树的高度至少为 (B)

- A. 10 B. 11 C. 12 D. 1011

18. 我们称质因数中只含有 3 和 5 的正整数为“好数”, 将所有好数从小到大排列组成数列 3,5,9,15,25,..., 那么第 10 个好数是 (C)。

- A. 75 B. 81 C. 125 D. 135

解析: 我们可以队列来解决这个问题, 我们记原数列对应的队列为 a。则 a 中的新元素一定

是从 $3*a$ 和 $5*a$ 这两个队列之中产生的。

a	3	5	9	15	25	27	45	75	81	125
3*a	9	15	27	45	75	81	135	225	243	375
5*a	15	25	45	75	125	135	225	375	405	625

因此第 10 个“好数”是 125。

19. 有一堆石子，初始有 2018 颗。小白和小蓝轮流取石子，每次最少取 1 颗石子，最多取 5 颗石子，取走最后一颗石子的一方获胜。小白先取石子，那么他第一次应该取走 (A) 颗石子，才能保证必胜。

- A. 2 B. 3 C. 4 D. 5

解析：经典博弈论问题，加入一开始石子的个数是 6 的倍数，那么后手必胜，因为先手如果取 x 个石子，那么后手对应取 $6-x$ 个石子即可。因此，小白一开始只要将剩下的石子调整为 6 的倍数，就可以获得胜利。

20. 全国青少年信息学奥林匹克竞赛的主办单位是：(D)

- A. 教育部 B. 科技部 C. 中国科协 D. 中国计算机学会

二、 阅读程序（程序输入不超过数组或字符串定义的范围：判断题正确填 T，错误填 F；除特殊说明外，判断题 2 分，选择题 3 分，共计 30 分）

1.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main(){
4      int n, k;
5      string s = "";
6      const string b = "0123456789ABCDEF";
7      cin>>n>>k;
8      while (n>0) {
9          s = b[n%k] + s;
10         n /= k;
11     }
12     cout<<s<<endl;
13     return 0;
14 }
```

判断题

- 1) 该程序的时间复杂度为 $O(\log_k n)$ 。(T)
2) 若该程序中输入 k 的值为 18，则输出结果为 n 的 18 进制表示。(F)

选择题

- 3) 输入 1: 13 2 输出 1: (A)
A. 1101 B. 1001 C. 1011 D. 1110
4) 输入 2: 2333 16 输入 2: (C)
A. 90D B. 90E C. 91D D. 91E

解析：这道题是经典的进制转换问题，将 10 进制数 n 转换为对应的 k 进制数。

- 1) 进制转换采用短除法，被除数 n ，除数 k ，时间复杂度为 $O(\log_k n)$ 。
2) 由于 b 数组只有 16 个字符，因此最大只能转换成 16 进制数， k 等于 18 时发生数组下标越界错误。

- 3) 将 13 转换为 2 进制数。 $(13)_{10}=(1101)_2$
 4) 将 2333 转换为 16 进制数。 $(2333)_{10}=(91D)_{16}$

2.

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  int a[1005],d[1005];
4  int find(int l , int r, int key) {
5      while (l<r) {
6          int m=(l+r)/2;
7          if (a[m]>key) r = m; else l = m+1;
8      }
9      return r;
10 }
11 int main() {
12     int n, len=1;
13     cin>>n;
14     for (int i=1; i<=n; i++) cin>>a[i];
15     d[1]=a[1];
16     for (int i=2; i<=n; i++) {
17         if (a[i]>=d[len]) {
18             len++; d[len] = a[i];
19         } else {
20             int j=find(1,len+1,a[i]); d[j]=a[i];
21         }
22     }
23     cout<<len<<endl;
24     return 0;
25 }

```

判断题

- 1) 若 $a[i]$ 大于数组 b 中的所有元素, 则第 20 行 `find()` 函数的返回值为 $len+1$ 。(T)
 2) 将第 9 行代码 “`return r;`” 改为 “`return l;`”, 不影响程序的运行结果。(T)

选择题

- 3) 该程序的时间复杂度为 (C)
 A. $O(\log_2 n)$ B. $O(n)$ C. $O(n\log_2 n)$ D. $O(n^2)$

4) 输入: 20

1 2 3 4 5 1 2 3 4 5 1 2 3 4 5 1 2 3 4 5

输出: (D)

- A. 5 B. 6 C. 7 D. 8

解析: 这道题是经典的二分查找求最长不下降子序列问题。

- 1) `find()` 函数的功能是找到 b 数组中第一个比 key 大的位置, 如果 key 大于 b 中所有元素, 那么返回值就是 $len+1$ 。
 2) 二分查找的循环条件是 `while(l<r)`, 因此当循环结束后 l 和 r 的值相等。
 3) 单次二分查找的时间复杂度为 $O(\log_2 n)$, 一共进行了 n 轮循环, 总的时间复杂度为 $O(n\log_2 n)$ 。
 4) 注意程序找到的是不下降子序列, 而不是单调递增子序列, 因此 1 1 1 1 这种形式也是合法的。通过观察, 最长不下降子序列为 “1 1 1 1 2 3 4 5”, 长度为 8。当然 “1 2 2

2 2 3 4 5”之类的也是可以的，但是根据程序只会找到字典顺序最小的一组解。

3.

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  int a[10005] ,b[10005];
4  int ans;
5  void merge_sort(int l, int r) {
6      if (l>=r) return;
7      int mid = (l+r) / 2;
8      int i = l;
9      int p = l , q = mid+1;
10     merge_sort(l, mid);
11     merge_sort(mid+1, r);
12     while(p<=mid || q<=r) {
13         if(q>r || (p<=mid && a[p]<=a[q])) {
14             b[i] = a[p]; i++; p++;
15         } else {
16             b[i] = a[q]; i++; q++;
17             ans += mid+1-p;
18         }
19     }
20     for(i = l ; i <= r; i++) a[i] = b[i];
21 }
22 int main() {
23     int n; ans = 0;
24     cin >> n;
25     for(int i = 1 ; i <= n; i++) cin >> a[i];
26     merge_sort(1, n);
27     cout << ans << endl;
28     return 0;
29 }
```

判断题

1) 将程序第 13 行条件中的“a[p]<=a[q]”改为“a[p]<a[q]”,不会影响最终结果。(F)

选择题

2) 该程序的时间复杂度为: (C) (2分)

A. $O(\log_2 n)$ B. $O(n)$ C. $O(n\log_2 n)$ D. $O(n^2)$

3) 输入 1: 6

3 1 6 5 2 4

输出 1: (B)

A. 6 B. 7 C. 8 D. 9

4) 输入 2: 55

1 1 2 1 2 3 1 2 3 4 ... 1 2 3 4 5 6 7 8 9 10

输出 2: (A)

A. 330 B. 335 C. 340 D. 345

解析：这道题是经典的归并排序求逆序对数量。

1) 如果将“a[p]<=a[q]”改为“a[p]<a[q]”,那么两个数在相等的情况下也会发生交换,这样就导致了最终的逆序对数量大于实际结果。

2) 归并排序合并两个数组的时间复杂度等于它们的长度之和,因此每一层递归的时间复杂

度都是 $O(n)$ 的，递归一共进行了 $\log_2 n$ 层，总的时间复杂度为 $O(n \log_2 n)$ 。

3) 序列 3 1 6 5 2 4 中一共 7 个逆序对，我们只需要统计每个数右边比它小的数有多少个就行了。

4) 可以按照 10 9 8 7 ... 3 2 的顺序去数数，数完以后把这些数从数列里面删掉就行了，因为如果删掉 10，那么 9 就是剩下的数里最大的一个了。

10 在最后面，没有逆序对。

9 有 2 个，最后一个没有逆序对，倒数第二个 9 后面又 8 个数，贡献为 8。

8 有 3 个，后面分别有 14 个、7 个、0 个数，贡献为 $7+14=21$ 。

同理 7 的贡献为 $6+12+18=36$ 。

6 的贡献为 $5+10+15+20=50$ 。

5 的贡献为 $4+8+12+16+20=60$ 。

4 的贡献为 $3+6+9+12+15+18=63$ 。

3 的贡献为 $2+4+\dots+14=56$ 。

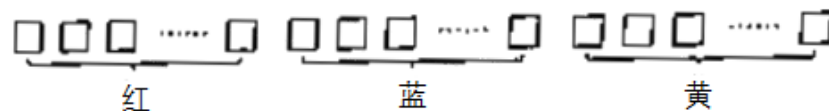
2 的贡献为 $1+2+\dots+8=36$ 。

总共 $8+21+36+50+60+63+56+36=330$ 个逆序对。

三、完善程序 (单选题, 每空 3 分, 共 30 分)

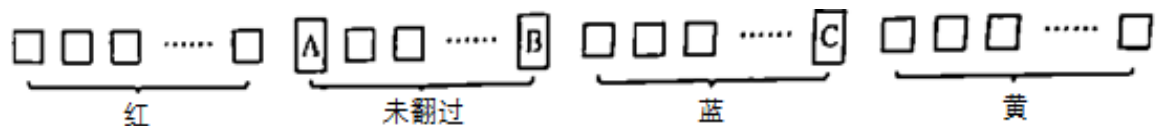
1.

(积木游戏) 有 n 个积木块排成一排，每个积木块向下的一面涂有红、黄、蓝三种颜色中的一种 (约定: 0 表示红色, 1 表示黄色, 2 表示蓝色)。游戏开始时，要求通过翻着与交换方式对积木块重新排列 (翻看的规则为每个积木块只能看一次)，最终使相同颜色的木块排列在一起，如下图所示：



设计一个翻看与交换的方案，实现上面的要求。

例如，设中间状态如下：



此时，从 A 处开始进行翻看交换：

若 A 为红色，则不用交换；

若 A 为蓝色，交换一次，即 A 与 B 交换；

若 A 为黄色，交换两次，即 A 与 B 交换一次，然后 B 与 C 再交换一次；

继续翻看交换下一个积木直到完成排列，游戏结束。

另外，初始状态 A 为第一块积木，B 和 C 为均最后一块积木。

【输入】

第一行为一个正整数 n ，表示积木的数量。

第二行为 n 个空格隔开的非负整数，表示初始积木的排列顺序，其中 0 表示红色，1 表示黄

色, 2 表示蓝色。

【输出】

一行一个整数, 表示交换的次数。

注意, 如果按照上述规则交换时两个积木的位置重合, 那么不需要计入总交换次数。

样例输入:

```
10
0 0 1 2 2 0 1 0 0 0
```

样例输出:

```
4
```

```
#include<bits/stdc++.h>
using namespace std;
int n,cnt,r,b,y;
int a[1010];
void swap(int &a, int &b) {
    if (①_____) {
        cnt--;
        return;
    }
    a = a+b; b = a-b; ②_____;
}
int main() {
    cin>>n;
    for (int i = 1; i <= n; i++) cin>>a[i];
    r = 1; b = n; y = n;
    while (③_____) {
        if (a[r] == 2) {
            swap(a[b],a[r]);
            b--; cnt++;
        } else if (a[r]==1) {
            ④_____;
            ⑤_____;
        } else r++;
    }
    cout<<cnt<<endl;
    return 0;
}
```

1) ①处应填: (C)

- A. a==b B. *a==*b C. &a==&b D. a && b

2) ②处应填: (B)

- A. a = a+b B. a = a-b
C. a = b-a D. b = b-a

3) ③处应填: (C)

- A. r<b B. r<y C. r<=b D. r<=y

4) ④处应填: (A)

- A. swap(a[b],a[r]);
 swap(a[b],a[y]); B. swap(a[r],a[b]);
 swap(a[r],a[y]);

C. swap(a[y],a[b]);
swap(a[y],a[r]);

D. swap(a[y],a[r]);
swap(a[y],a[b]);

5) ⑤处应填: (D)

A. y--; cnt++;

B. y--; cnt+=2;

C. r++; y--; cnt+=2;

D. b--; y--; cnt+=2;

解析: 这道题是一道模拟题, 考察的是阅读理解的能力。

1) 我们如果注意到“如果按照上述规则交换时两个积木的位置重合, 那么不需要计入总交换次数”这句话, 就会知道这个地方是要把两个积木重合的情况给排除掉。两个积木重合相当于他们指向同一块内存地址, 因此答案为&a==&b。

2) 不借助第三个变量交换两数的操作: a=a+b;b=a-b;a=a-b;

3) b 是下一块蓝色积木放置的位置, 因此 b+1 之后的积木都是被看过的, 同理 r 左边的积木也都被看过一遍了, 因此循环执行的条件为 r<=b。

4) 根据“若 A 为黄色, 交换两次, 即 A 与 B 交换一次, 然后 B 与 C 再交换一次”, 先交换 a[r]和 a[b], 再交换 a[y]和 a[b]。

5) 交换后黄色积木多了一块, 位置向左移动一格, 蓝色积木为了给黄色积木腾位置, 也要向左移一格, 进行了两次交换, cnt 的值加 2。

2.

(数据分组) 给定 n 个正整数, 将它们分组, 使得每组中的任意两个数互质 (它们的最大公约数为 1)。按照以下算法可以得到最少的组数:

第一步: 将第 1 个整数分到第 1 组;

第二步: 尝试将第 2 个至第 n 个整数分到已有的分组中, 若能分到已有的分组中, 则分到第一个符合条件的组; 若不能分到已有的组, 则分到新生成的组中。

经过回溯算法, 可以找到最小的分组数量。

例如对“70, 99, 25, 54, 11, 100”这 6 个整数分组, 具体分组情况如下表所示:

组别	第 1 组							
位置	0	1	2	3	4	5	6	
值	2	70	99	0	0	0	0	
组别	第 2 组							
位置	7	8	9	10	11	12	13	...
值	3	25	54	11	0	0	0	...

【输入】

第一行为 n, 表示数据的个数

第二行为 n 个空格隔开的正整数, 表示分组的数据。

【输出】

一行一个正整数, 表示最小分组数量。

样例输入:

6

70 99 25 54 11 100

样例输出:

3

```
#include<bits/stdc++.h>
using namespace std;
int n,m,ans;
int a[51];
int b[51][51];
int gcd(int m, int n) { //求最大公约数
    if (n==0) return m;
    else return ①_____ ;
}
void find(int i) {
    if (i==n) {
        if (ans>m) ans = m;
        return;
    }
    for (int k = 0; k<m; k++) {
        bool flag = true;
        for (int j = 1; j<=b[k][0]; j++)
            if (②_____) {
                flag = false;
                break;
            }
        if (flag) {
            b[k][0]++;
            ③_____ ;
            find(i+1);
            b[k][0]--;
        }
    }
    ④_____ ;
    find(i+1);
    m--;
}
int main() {
    cin>>n;
    for (int i = 0; i < n; i++) cin>>a[i];
    ⑤_____ ;
    b[0][0] = 1; b[0][1] = a[0];
    find(1);
    cout<<ans<<endl;
    return 0;
}
```

1) ①处应填: (D)

A. gcd(m,m/n) B. gcd(m,m%n) C. gcd(n,m/n) D. gcd(n,m%n)

2) ②处应填: (C)

A. gcd(a[i],b[j][k])>1 B. gcd(a[i],b[j][k])==1
C. gcd(a[i],b[k][j])>1 D. gcd(a[i],b[k][j])==1

3) ③处应填: (B)

- A. `b[k][b[k][0]]=i`
- C. `b[k][b[k][0]+1]=i`

- B. `b[k][b[k][0]]=a[i]`
- D. `b[k][b[k][0]+1]=a[i]`

4) ④处应填: (D)

- A. `m++;`
`b[m][0] = 1;`
`b[m][1] = i;`
- C. `b[m][0] = 1;`
`b[m][1] = i;`
`m++;`

- B. `m++`
`b[m][0] = 1;`
`b[m][1] = a[i];`
- D. `b[m][0] = 1;`
`b[m][1] = a[i];`
`m++;`

5) ⑤处应填: (C)

- A. `m = 0; ans = n;`
- C. `m = 1; ans = n;`

- B. `m = 0; ans = 0;`
- D. `m = 1; ans = 0;`

解析: 这道题是一道搜索题, 考察了回溯算法。

1) 利用辗转相除法求最大公约数, 这里考查了递归的写法。

2) `flag` 表示新数是否与原来的数互质, 后文中 `flag` 变为 `false`, 说明这里不互质, 即两个数的最大公约数大于 1, 因此答案为 `gcd(a[i], b[k][j]) > 1`。

3) `b[k][0]` 表示的是第 `k` 组总共有几个数, 因此新放进去的数的位置是 `b[k][b[k][0]]`。

4) 根据上下文分析此处应该是新建立一个分组的情况, 因此先对 `m` 加一, 然后更新 `b[m][0]` 和 `b[m][1]` 的值。

5) 一开始第一个数已经被强制分到第一组了, 因此 `m` 等于 1, 然后答案的初始值可以设定为 `n`, 因为可以每个数都各自分成一组。